CS3.301: Operating Systems and Networks
Lecture 5: CPU Scheduling

INSTRUCTOR: DR. DEEPAK GANGADHARAN

Need for high level OS policies (Scheduling) – A parallel

- Multiple packaging workstations in a factory
- Limited robots carry materials from storage units to packaging workstations
- How will you schedule the picking of materials from storage units to workstations by robots such that some properties are optimized?
- Maximize throughput, minimize the maximum time required to process all the products
- Same is the situation within a computing system!

What we need to know to ensure good policy?

- How many products are there?
- What are the categories of each product?
- How much time does it take for each product to be processed?
- How frequently are the product orders coming in?

What does it mean?

- For scheduling, we need an idea of the **workload**
  - Assumptions of processes running in the system
    - Number of processes
    - RAM required
    - CPU utilization
    - Any Input/Output, if yes what kind?

Some workload assumptions

- Each process that is ready/needs to be executed and those executing called **Job**!
- Assumptions
  - Each job runs for **same amount of time**
  - All jobs arrive at the same time
  - Once started, each job runs to completion.
  - All jobs **only use the CPU** (No I/O)
  - **Run time** or execution time of each job is **known**

How good is the policy?

- Metrics are used to measure the goodness of a policy
- **Performance metric**:
  - Turnaround time – Time difference between job completion time and the arrival time

$$T_{turnaround} = T_{completion} - T_{arrival}$$

  - Another metric is fairness – How fair is the scheduling?
    - Jain's Fairness index – Measures how evenly a resource is shared among a group of users

$$J = \frac{\left(\sum_{i=1}^{n} x_i\right)^2}{n \sum_{i=1}^{n} x_i^2}$$

      where n is the number of users (jobs in this case) and x_i is the allocation to each user
    - If $J = 1$, every user gets exact same amount of resource
    - If J is near 0 or low value, then only few users get all the resources
    - May not go hand in hand with performance

Scenario 1: All assumptions intact

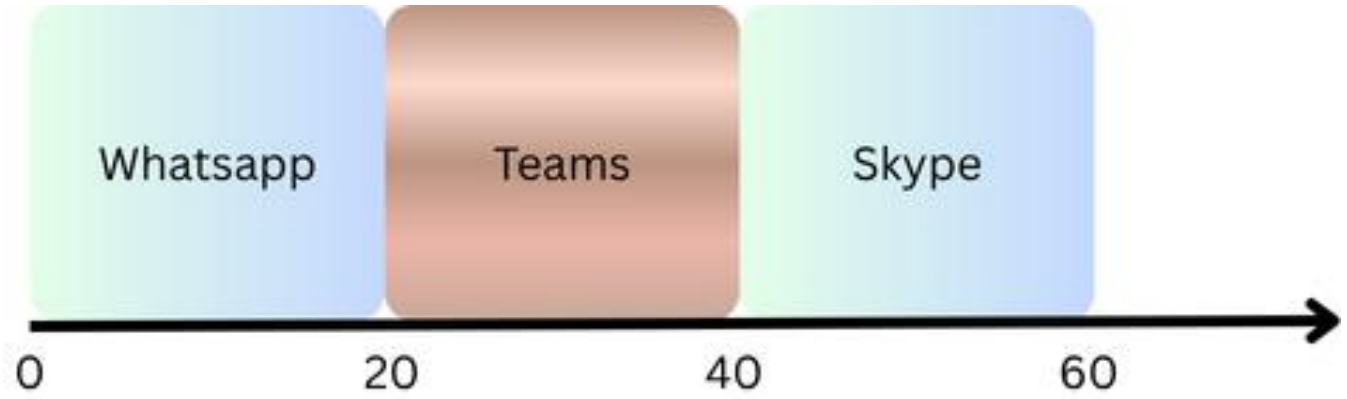
- Imagine 3 jobs – Whatsapp, Skype and Teams update arriving at the same time
- Each of them taking same time to complete

Process	Arrival	Time to Complete
Whatsapp (w)	~0	20
Skype (S)	~0	20
Teams (T)	~0	20

- How to go about this?

First Come First Serve (FCFS) Policy

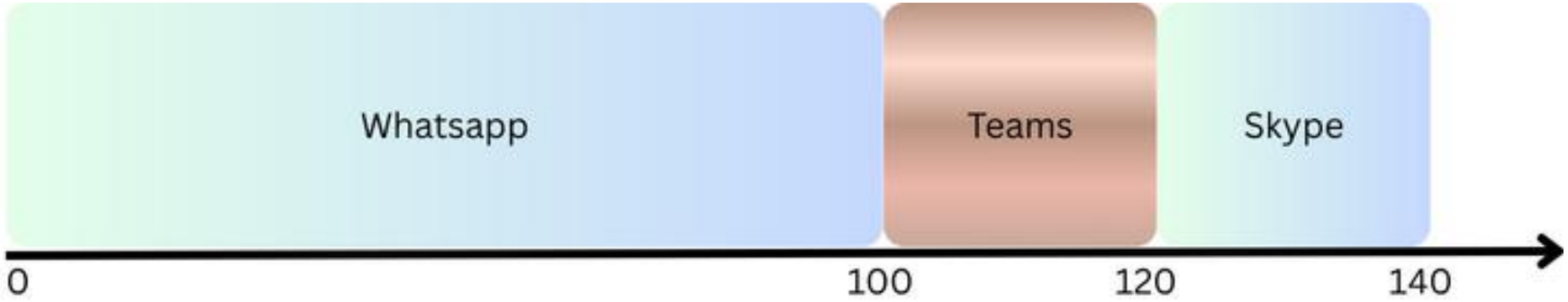
- Most basic algorithm a scheduler can implement
- First one gets scheduled



- Assume that they arrive at the same time. Example t = 0
  - For sake of simplicity, W arrived just a hair before T which arrived just a hair before S
- Average turnaround time = (20 + 40 + 60)/3 = 40

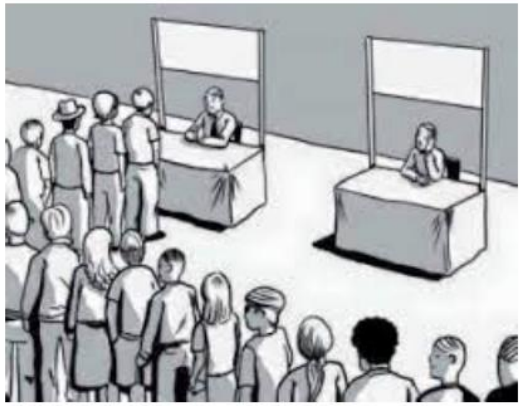
What if each job no longer runs for the same time?

Relaxing Assumption 1



- Average turnaround time = (100 + 120 + 140)/3 = 120

FCFS is not that great



- Waiting time can go very high
  - Convoy effect
  - Think about waiting in line to buy just one item
- Can there be a better algorithm?

What if each job said what time it requires!

Shortest Job First (SJF) Policy

- Idea originating from Operations research
- Policy: Run the shortest job first

Process	Arrival	Time to Complete
W	0	100
S	0	20
T	0	20

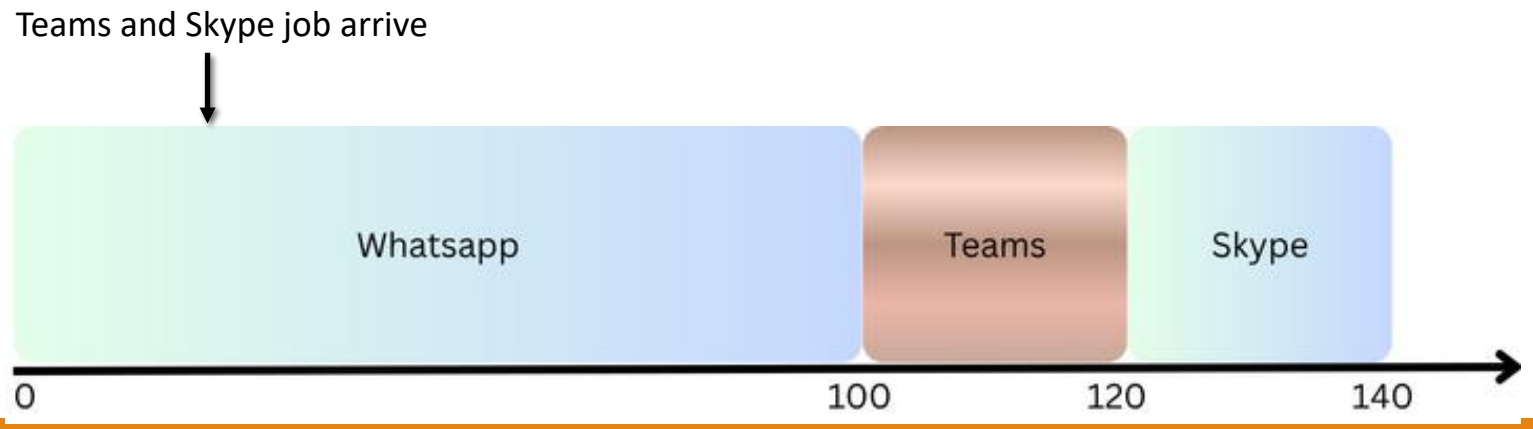
SJF Policy

Average turnaround time = (20+40+140)/3 = 66.3

- Almost a factor of 2 improvement!
- Under the assumption that all jobs arrive at the same time, it can be proved that SJF is an optimal scheduling algorithm (with respect to average turnaround time)
- What if the jobs arrive at different times?

SJF Policy – Different arrival times

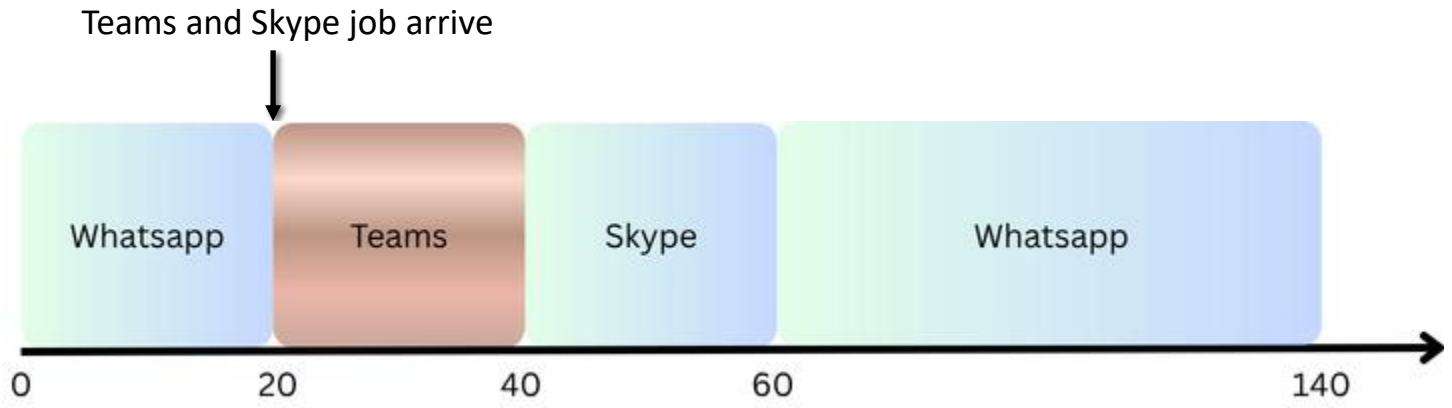
- Whatsapp job arrives first
- Teams and Skype job arrive at around t = 20
- Average Turnaround time = (100+100+120)/3 = 106.6
- Suffer the same convoy problem!
- What better can a scheduler do?



Shortest Time to Completion First (STCF)

- Relax assumption 3 – i.e., jobs must run to completion
- SJF by our definitions is a non-preemptive scheduler.
- Based on discussion of context switching, scheduler can do something else when jobs Teams and Skype arrive – it can preempt the job Whatsapp.
- Add preemption to SJF → Shortest Time to Completion First or Preemptive SJF
- Scheduling Policy: Any time a new job enters the system,
  - Determine which of the remaining jobs (including the new job) has the least time left
  - Schedule that job

Shortest Time to Completion First (STCF)



Average turnaround time = ((140-0) + (40-20) + (60-20))/3

= 66.3

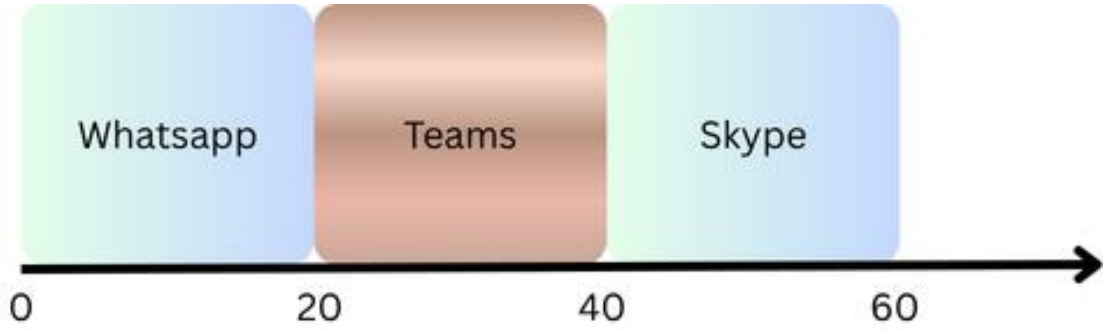
STCF is provably optimal under the new assumptions!

A new metric: Response Time

- Known job lengths + Jobs only used CPU + Turnaround time metric → STCF is a great policy!
- STCF made sense for early Batch computing systems.
- Different scenario in time shared machines → Users demand interactive performance
- A new metric was thus proposed → Response Time
- Response time is defined as

$$T_{response} = T_{firstrun} - T_{arrival}$$

STCF – Response Time



Arrival time of each job = 0. Execution time of each job = 20.

Response time of Whatsapp = 0 – 0 = 0, Response time of Teams = 20 – 0 = 20, Response time of Skype = 40 – 0 = 40

Average turnaround time = (20+40+60)/3 = 40

Average response time = (0 + 20 + 40)/3 = 20

When three jobs arrive at the same time, the third has to wait until the previous two jobs finish before being scheduled just once → Not ideal

Not good for response time and interactivity.

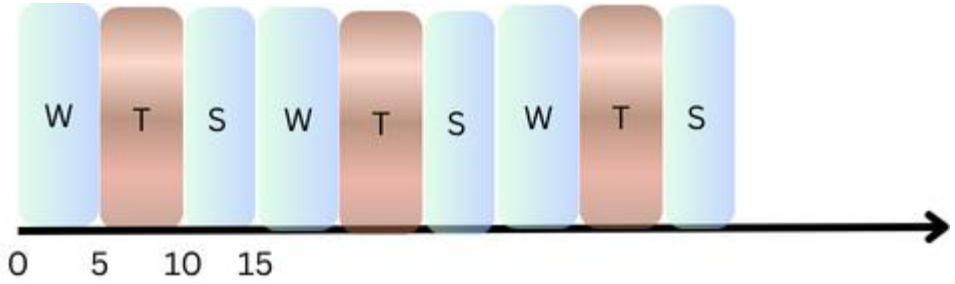
Can we do better? Can we build a scheduler which is sensitive to response time?

Round Robin (RR) Scheduling

- Instead of running jobs to completion, RR runs a job for a **time slice** (also called **scheduling quantum**)
- **Policy**
  - Run a job for a time slice
  - Switch to next job from the front of the ready queue
  - Repeat till all are finished executing
- Time slicing enabled using *timer interrupts*
- Length of a time slice = Multiple of timer interrupt period
  - Example: If timer interrupts every 20 millisecond, time slice could be 20, 40, 60 etc.

RR scheduling

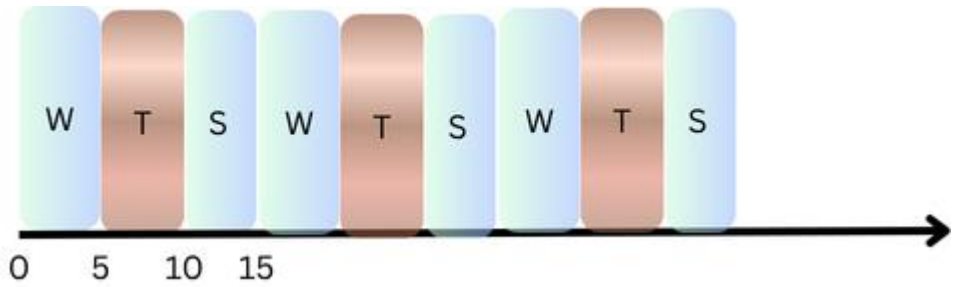
- What if we do round robin with a time slice = 5 sec?
- Average response time = (0 + 5 + 10)/3 = 5
- Length of the time slice critical for RR
  - Shorter the time slice → Better the response time



- Can we reduce the time slice as much as possible? → Do you see any issue?

RR scheduling

- Time slice = 5 sec
- Average response time = (0 + 5 + 10)/3 = 5
- Too small a time slice can result in overhead because too much context switch
- RR is good if we can find an optimal time slice
- What about turn around time? → Not so good



Trade-Offs

- Turnaround time only cares about completion
  - No consideration of fairness
- Key aspect is to consider the trade-offs
  - Do you want responsiveness or fairness?
  - Do you want to consider security or performance?

Thank You!!